

PilotFish Directory Listener – Complete Options and Behavior Reference (26R1.11 WAR)

Derived from decompiled eip.war.hs.26R1.11

July 24, 2026

At a glance (plain English)

Think of this as a **folder watcher**. You point it at a directory on disk. When new files show up (and pass your filters), it picks them up and starts an integration transaction for each one—so the rest of your route can transform and deliver the data. After pickup you can move, delete, or keep the original file.

Directory Listener — what it does

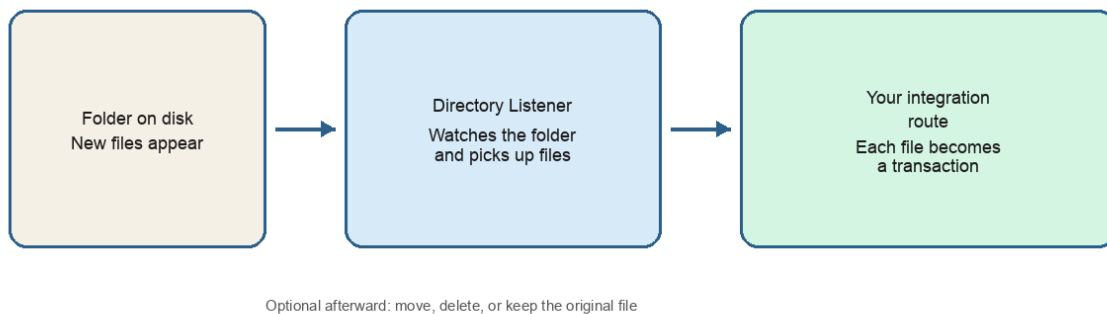


Figure 1: At a glance (plain English)

- **Watches** a folder on a schedule (polling).
- **Selects** files by name, extension, age, and other rules.
- **Starts work** in PilotFish for each matching file.
- **Cleans up** the file afterward if you configure Move or Delete.

Purpose

This document is a **deep dive** into the **Directory / File** listener as shipped in **eip.war.hs.26R1.11**: how it polls, which files it accepts, how it builds transactions, how post-processing works, and every configuration option – including dependencies, defaults, and subtle edge cases recovered from decompiled bytecode.

It is intentionally the same *depth* as the Git-sourced reference ([PilotFish-Directory-Listener-Reference.pdf](#)), but **version-stamped from the WAR** so it matches what customers run.

Provenance	Value
WAR	<code>eip.war.hs.26R1.11</code>
Module JAR	<code>xcs-modules-file-1.0.1.jar</code>
Core JAR	<code>xcs-core-1.2.6.jar</code>
Decompiler	CFR 0.152
Primary class	<code>com.pilotfish.eip.modules.file.DirectoryLi</code>
Decompiled path	<code>war-pipeline/decompiled/eip-26R1.11/.../Di</code>
Listener type string	<code>Directory / File</code>
Description	Monitors a specified directory for new files.
Help ID	<code>console.listener.directory</code>
Category	FILESYSTEM

Related classes (same WAR):

Class	Role
<code>DirectoryListener</code>	Polling, filtering, attribute creation, dispatch
<code>AutoDeleteFileInputStream</code>	Delete or move source file when the input stream is closed
<code>AbstractListener</code>	Base options (restart, CLI, FIFO, throttling, inactivity, scheduling)
<code>DirectoryUtils</code>	Retry helpers for delete/move

Optional UI figures: attach version-matched eiConsole screenshots via the freeform capture inbox when available. This WAR-derived deep dive does not depend on screenshots.

Bytecode recovers labels, defaults, tooltips, dependencies, and method behavior. Original Java comments / FIXMEs may be absent; narrative below is inferred from executable logic in the decompile.

1. High-level behavior

1. On **engage**, a **single-thread scheduled executor** (`NTMThreadFactory`) runs `doListening()` at a fixed rate equal to the **Polling interval** (seconds). First run is delayed by **1 second**.

2. Each poll:
 - Initializes listener `TransactionData` context
 - Validates synchronous / max-transactions constraints
 - Either picks up a **single full path** (MVS / full-path mode) or lists a **polling directory**
 - Filters candidates via `FileFilter.accept`
 - Optionally sorts by modified time
 - Caps how many files are taken this cycle (**Max Transactions Per Run**)
 - Recurses into subfolders if enabled (after processing files in the current folder, subject to remaining quota)
 - For each accepted file: copy into EIP cache, set attributes, dispatch into the route, then post-process (delete / move / keep) when the stream closes
3. On **disengage**, the scheduler is shut down.

Critical design point: Post-process (delete/move) is tied to `AutoDeleteFileInputStream.close()`, which runs after the file contents have been copied into the cache used to start the transaction – **not** after the entire route completes. “Delete after processing” means “after the listener has successfully read the file into a transaction stream,” not “after the transport succeeds.”

2. Polling lifecycle

2.1 Engage / disengage

```
engageListener()
    pollingInterval = manager.getIntValue(PollingInterval)
    scheduledExecutor = newSingleThreadScheduledExecutor(NTMThreadFactory)
    scheduleAtFixedRate(doListening, initialDelay=1s, period=pollingInterval)

disengageListener()
    scheduledExecutor.shutdown()
```

Nuance	Detail
Invalid polling interval	Logs error and <code>notifyListenersOnError</code> ; listener does not schedule
Single thread	Polls never overlap on this executor; a slow <code>doListening</code> delays the next cycle
Fixed rate	Uses <code>scheduleAtFixedRate</code> – periods are measured from start-to-start; overruns cause catch-up pressure

2.2 `doListening()` outline

1. `initializeData()` – ensure listener `TransactionData`, set originating/current listener
2. `checkRequiredData(...)`
3. Enforce: if **Synchronous** is true, **Max Transactions Per Run** must be **1** (else `IllegalArgumentException`)
4. Branch:

- **Specify full file path = Enabled** -> process that one file if it exists
 - Else -> resolve polling directory; validate; optionally ensure move target exists; `listenOnDirectory`
5. Any exception -> `notifyListenersOnError(..., "Unexpected error while listening.")`

3. Configuration reference (DirectoryListener-specific)

Items below come from `DirectoryListener.getConfigurationDescriptor()`. Tabs are UI groupings (`ConfigurationTab`).

3.1 Basic / polling

Polling interval

Property	Value
UI label	Polling interval
Tag	<code>PollingInterval</code>
Type	Time (seconds)
Default	30
Tooltip	How often the directory is polled for new files
Required	Yes

Nuances:

- Units are **seconds** (passed to `TimeUnit.SECONDS`).
- Combined with initial delay of 1s on engage.
- Very small intervals increase filesystem load and lock contention on busy shares.

Polling directory

Property	Value
UI label	Polling directory
Tag	<code>PollingDirectory</code>
Type	Path (directories only)
Default	""
Tooltip	The directory path to be polled
Required	Yes (when full-path mode is Disabled)

Nuances:

- Resolved via `ConfigurationManager.getFile(..., loader)` – may return **null** if unresolved.
- After resolve, path is re-wrapped as `new File(file.getAbsolutePath())` (PLIT-881: partial folders relative to route).

- Must be an existing directory when polled; otherwise behavior depends on **Stop if directory not found**.
- Hidden when **Specify full file path** is Enabled.

File/folder name restriction

Property	Value
UI label	File/folder name restriction
Tag	FileNameRestriction (constant name FileNameRestrictions_TAG)
Type	String
Default	"" (no restriction)
Tooltip	An expression which must be contained by a file name to be picked up

Acceptance logic (accept):

- Applies to the **base file name** (via `Util.getFileName(name)`), **not** the extension.
- File is accepted if either:
 - the name **contains** the restriction string (`indexOf`), **or**
 - the name **matches** the restriction as a **regex** (`String.matches`)
- Empty restriction -> no name filter.

Nuances:

- This is **not** “exact match only.” Substring containment is enough.
- Regex uses full-string **matches** (must match the entire name), while substring uses `indexOf`.
- Directories always pass **accept** (see §5) so name restriction does not block folder recursion discovery – only files are filtered by name/extension/age.

File extension restriction

Property	Value
UI label	File extension restriction
Tag	FileExtensionRestriction (constant name FileExtensionRestrictions_TAG)
Type	List / comma-separated string
Default	""
Tooltip	Valid filename extensions

Acceptance logic:

- Parsed with `StringTokenizer` on commas; each token trimmed and **lowercased**.
- Compared to `Util.getExtension(name)`.
- Empty -> all extensions allowed.

Nuance: Configure extensions without a leading dot in practice (matching whatever `Util.getExtension` returns). Case of the config entries is normalized to lowercase.

Specify full file path (MVS / single-file mode)

Property	Value
UI label	Specify full file path
Tag	UseFullFilePath
Choices	Disabled (default), Enabled
Tooltip	If enabled, a full file path may be specified to pick up a particular file

Dependencies:

When	Shown / used
Disabled	Polling directory, name restriction, extension restriction
Enabled	Path to file/folder only

Path to file/folder

Property	Value
UI label	Path to file/folder
Tag	FullPathToFile (constant name FullFilePath_TAG)
Type	Path (files only)
Default	""

Behavior when Enabled:

- Builds **File** from the string value (not necessarily via route-relative **getFile** helper used for polling directory).
- If file exists -> **processFile(file, 1)**.
- If missing -> error log and **IOException** ("File doesn't exist").

Nuance: Cycle size is hard-coded to 1. Subfolder iteration and directory listing are skipped.

3.2 Post-process tab

Postprocess operation

Property	Value
UI label	Postprocess operation
Tag	PostProcessOperation
Choices	(<i>unconfigured empty</i>), Delete, Move, Keep
Default	Unconfigured empty string
Unconfigured sentinel	""

Choice	Meaning in code
Delete	<code>AutoDeleteFileInputStream</code> with <code>moveDir=null</code> , <code>keep=false</code> -> delete on close
Move	Stream constructed with target move directory; rename/copy on close
Keep	<code>keep=true</code> -> close does not delete or move
Unconfigured	Treated like non-Keep / non-Move for stream construction: delete path if <code>keep</code> is false and <code>moveDir</code> is null

When post-process runs: On `AutoDeleteFileInputStream.close()` after contents are copied into the EIP cache – **before / independent of** downstream route success.

Dependencies: Target directory + compat flag shown only when operation is **Move**.

Target directory

Property	Value
UI label	Target directory
Tag	<code>TargetDirectory</code>
Type	Path (directories only), writeability check enabled
Tab	Post-process

Resolution (`getMovedDir`):

Compat mode	Resolution
Use Pre-19R1 Relative Path Behavior = true	<code>new</code> <code>File(manager.getStringValue(TargetDirectory))</code> – relative to installation / process CWD semantics , not route loader
false (modern)	<code>manager.getFile(TargetDirectory, ..., loader)</code> – route-aware resolution

On poll (directory mode + Move):

- If target missing, listener attempts `mkdirs()`.
- If still not a directory and **Stop if directory not found** is true -> throw / stop path.
- If stop flag is false -> log and return without processing.

Move implementation (`AutoDeleteFileInputStream.moveFile`):

1. Destination name via `Util.getNewFileName(originalName, moveDir)` (avoids collisions by generating a new name when needed).
2. `DirectoryUtils.moveFileWithAttempts(..., 10)` retries.
3. If source still exists and move failed -> copy via streams; on failure, log error (delete fallback messaging appears in related helpers).

Use Pre-19R1 Relative Path Behavior

Property	Value
UI label	Use Pre-19R1 Relative Path Behavior
Tag	CompatModeMoved
Default	false in descriptor
Tooltip	Relative Target Directory paths resolve from installation directory rather than Route directory (pre-19R1)

Critical load nuance (readConfiguration):

- If the saved XML **does not contain** a CompatModeMoved element, the listener **forces** CompatModeMoved = true so older routes keep pre-19R1 path behavior.
- If the element exists, the stored value is honored.

Implication: Brand-new configs default false in the UI descriptor, but **routes never re-saved on 19R1+** behave as compat=true even though the descriptor default is false.

3.3 Criteria tab (file age / stability)

Min seconds since file modified

Property	Value
UI label	Min seconds since file modified
Tag	MinSecondsSinceFileModified
Default	10
Range	0-65000
Tooltip	Minimum number of seconds elapsed since file modified

Logic (gpmAccept): Reject if $0 \leq (\text{now} - \text{lastModified}) < \text{minSeconds} * 1000$.

Nuances:

- Purpose: avoid picking up files still being written.
- Comment in source: **not a valid check when polling a remote directory** where clocks / lastModified semantics differ (SMB/NFS quirks).
- Default 10 seconds is a common source of “why isn’t my file picked up immediately?”

Min days since file modified

Property	Value
UI label	Min days since file modified
Tag	MinDaysSinceFileModified
Default	-1 (disabled)

Property	Value
Range	-1-65000

Reject if $(\text{now} - \text{lastModified}) / 86400000 < \text{minDays}$ when $\text{minDays} \neq -1$.

Max days since file modified

Property	Value
UI label	Max days since file modified
Tag	MaxDaysSinceFileModified
Default	-1 (disabled)

Reject if day difference $> \text{maxDays}$ when enabled.

Nuance: Day math uses integer division of milliseconds by 86400000 – coarse day buckets, not calendar-aware.

3.4 Sorting tab

File Sorting Method

Choices	Default
System Default, Modified Time	System Default

Sorting Direction

Choices	Default
Ascending, Descending	Ascending

Behavior:

- **System Default:** no sort – order is whatever `File.listFiles` returned.
- **Modified Time + Ascending:** `LastModifiedFileComparator.LASTMODIFIED_COMPARATOR`
- **Modified Time + Descending:** `LASTMODIFIED_REVERSE`
- Unknown method/direction -> `EIException`

Processing order nuance: After filtering into `acceptedFiles`, the listener iterates **from the end of the list toward the start** (for `i = size-1; i >= 0`). Combined with sort direction, this means the **effective pickup order is the reverse of the sorted list order**. When debugging “oldest first,” verify both sort settings **and** this reverse iteration.

3.5 Advanced tab (DirectoryListener)

Stop if directory not found

Property	Value
UI label	Stop if directory not found
Tag	StopWhenPollingFailure
Default	true
Tooltip	Listener will stop and report an error when polling directory, or postprocess target directory do not exist

When **false**: missing/unreadable polling or target dirs are logged and the poll returns quietly (no throw for those paths).

Inserted in the descriptor after **Restart on listening error** (base Advanced item).

Max Transactions Per Run

Property	Value
UI label	Max Transactions Per Run
Tag	SerializedTransactionsTag
Default	250
Range	1-255

Despite the UI name “transactions,” the quota is applied to accepted files per poll cycle (before tokenization). Tokenizers can turn one file into many transactions afterward.

Mechanics:

- **serializedTxLeft** reset each poll to this value.
- In each directory, files are accepted until the remaining quota is exhausted.
- Subfolder recursion continues with **remaining** quota (**listenOnDirectory** returns immediately if **serializedTxLeft** <= 0).

Synchronous dependency (PLIT-4807): Synchronous UI controls are only available when this value is **1**. Runtime also enforces the same rule.

Include sub-folders

Property	Value
UI label	Include sub-folders
Tag	SubFolderIterationTag
Default	false

When true, after processing files in a directory, each readable subdirectory is scanned recursively with the same filter/quota.

Nuances:

- Directories always **accept** as true (subject to hidden-file rules).
- Unreadable directory -> error log and **return from listenOnDirectory** (stops further processing in that call – aggressive failure).
- Full folder path restriction is the intended way to limit which subtrees contribute files.

Full folder path restriction

Property	Value
UI label	Full folder path restriction
Tag	FullFolderPathRestrictionsTag
Default	""
Tooltip	Regex that must be contained (via <code>Matcher.find</code>) by the file's absolute path

Nuances:

- Uses `Pattern.compile + matcher.find()` – **substring regex search**, not full-match.
- Applied to files in **accept**; typically meaningful with Include sub-folders.
- Invalid regex will throw at accept time.

Include hidden files

Property	Value
UI label	Include hidden files
Tag	HiddenFilesTag
Default	false

If false, `File.isHidden()` -> reject (files and directories).

Combine files into one stream

Property	Value
UI label	Combine files into one stream
Tag	CombineFiles
Default	false
Tooltip	Reads in all matching files as a single stream

When true, all accepted files in the **current directory batch** are concatenated into one cache; one dispatch at the end. Attributes come from the **first** file in the combine loop.

Number of header lines

Property	Value
UI label	Number of header lines
Tag	<code>HeaderLines</code>
Default	0
Range	0-1000
Depends on	Combine files = true

Behavior:

- First file: header lines included.
- Subsequent files: that many lines skipped via `BufferedReader.readLine()`.
- Uses character Writer path when `headerLines > 0`; otherwise raw byte copy.

Nuance: Line-oriented skipping implies text; binary combine should keep header lines at 0.

Synchronous

Property	Value
UI label	Synchronous
Tag	<code>SynchResponse</code>
Default	<code>false</code>
Tooltip	Wait for synchronous response before next file; Max Transactions Per Run must be 1

When true:

- Uses `dispatchForSynchronousResponse` with timeout.
- Null response -> `notifyListenersOnError` ("Synchronous response timed out").
- Non-null stream is closed (response body not consumed by the listener beyond wait completion).
- **Tokenizers disabled** (descriptor dependency); runtime also throws if tokenizers set with sync.

When false: Tokenizers field is available; async `dispatchListeners`.

Synchronous Timeout

Property	Value
UI label	Synchronous Timeout
Tag	Timeout (shared <code>AbstractConfiguration</code> timeout tag)
Default	60
Units	Seconds
Min	1

Tokenizers

Property	Value
UI label	Tokenizers
Tag	Tokenizers
Default	""
Visible when	Synchronous = false

Splits one file into multiple transactions when non-empty.

Tokenizer value	Behavior
Empty	One transaction for whole file
NEWLINE	Treated as delimiter "\r\n" via StringTokenizer
Other non-LENGTH string	Used as delimiter charset for StringTokenizer
LENGTH n	Fixed-width records of length n (substring stepping; see source for endPos off-by-one style bound)

Per-token attributes:

- `com.pilotfish.BatchId` – 1-based index of non-empty token
- `com.pilotfish.BatchSize` – `MutableInt` incremented under synchronization; final batch size logged at DEBUG

Empty/whitespace-only tokens are skipped (`trim().length() > 0`).

Nuances:

- Tokenization loads the **entire file into a byte[] / String** – large files mean large memory use.
- Each token calls `notifyListeners(bytes, atts.cloneAllForResponse())`.
- Incompatible with Synchronous mode.

3.6 Scheduling tab

From `getSchedulerConfigurationDescriptor()` (inherited pattern):

Item	Tag	Purpose
Scheduled start time	Scheduler start tag	Advanced schedule for when listening is allowed
Scheduled end time	Scheduler end tag	End of scheduled window
Weekly Days to exclude	Exclude days	Skip certain weekdays
Dates to exclude	Exclude dates	Holiday / specific date skips

Item	Tag	Purpose
Time Zone	Time zone tag	Defaults to console TZ at initial configuration

Scheduling gates when the listener is considered active relative to calendar rules (base listener/scheduler infrastructure), independent of the polling interval itself.

4. Inherited AbstractListener options (from xcs-core in this WAR)

DirectoryListener extends AbstractListener and inherits base items via `super.getConfigurationDescriptor().getBaseConfiguration()` in decompiled AbstractListener.

4.1 Advanced (base)

Label	Tag	Default	Tooltip (from bytecode)
Initialize on trigger only	IS_TRIGGERABLE_LISTENER	true	Listener doesn't start until a trigger initializes it
Allow command-line invocation	CLI allowed tag	false	Can be invoked using the CLI client
Restart on listening error	RESTART_ON_ERROR_TAG	true	Restart after listen errors
FIFO Queue Name	FIFOQueueName	Empty	Empty disables FIFO
FIFO Queue Delay	FIFOQueueDelay	500	Delay between FIFO queue polls

4.2 Throttling (base)

Label	Tag	Default / choices
Throttling Mode	ThrottlingMode	None / Timed / Concurrent (default index 0 = None)
Throttling Mechanism	ThrottlingMechanism	Blinking / Queued
Max Concurrent Messages	ThrottlingConcurrentMessages	1
Timed Emission Interval	ThrottlingTimedInterval	1000
Synchronous Timeout Interval	ThrottlingSynchronousTimeout	60000

Dependencies: Timed mode shows mechanism, concurrent messages, timed interval. Concurrent mode shows mechanism, concurrent messages, and throttling synchronous timeout. Concurrent mode expects a Throttling Response Processor later in the route.

Throttling limits how fast/how many transactions the listener **emits into the framework**, separate from Max Transactions Per Run (which limits how many **files** are read per poll).

4.3 Inactivity monitoring (base)

Label	Tag	Default
Enable Inactivity Monitor	<code>CheckInactivity</code>	false
Min. Transactions to Expect	<code>InactivityTransactionCount</code>	1
Monitoring Interval	<code>InactivityPolling</code>	500
Times to Monitor	<code>MonitoringTime</code>	time-of-day (builder default)
Days to Exclude from Monitoring	<code>InactivityExcludedDays</code>	"
Include Errors in Transaction Count	<code>InactivityIncludeErrors</code>	false

When enabled, dependent fields appear. If expected volume is not met in the interval, the listener raises a non-transaction error path (`notifyListenersOnError`).

4.4 Scheduling (from `getSchedulerConfigurationDescriptor`)

Label	Tag
Scheduled start time	<code>SchedulerStartTag</code>
Scheduled end time	<code>SchedulerEndTag</code>
Weekly Days to exclude	<code>ExcludeDaysTag</code>
Dates to exclude	<code>ExcludeDatesTag</code>
Time Zone	<code>TimeZone</code> (default display: System Default / console TZ at first config)

5. File acceptance algorithm (accept + `gpmAccept`)

Order of checks for a candidate File:

1. **Hidden:** if Include hidden files is false and `isHidden` -> reject
2. **Directory:** always accept (so recursion can see it)
3. **Name restriction:** substring or full regex match on base name
4. **Full folder path restriction:** regex find on absolute path

5. **Extension restriction:** must be in comma list (lowercased)
6. **Min/max days since modified**
7. **gpmAccept:**
 - Reject if **not writable** (!canWrite) – read-only files are skipped so delete/move won't fail later
 - Reject if modified too recently (min seconds)
 - Reject if listed in **failedDeleteFiles** (legacy fingerprint set – see known issues)

Additionally, when building the accepted list in **listenOnDirectory**, each file must **canRead()** and **canWrite()** or the method logs an error and **returns** (aborts that directory pass).

6. Transaction attributes produced

Registered (design-time metadata) and set at runtime in **createTransactionAttributes**:

Attribute key	Type	Meaning
<code>com.pilotfish.FileName</code>	String	Base name without extension
<code>com.pilotfish.FileExtension</code>	String	Extension after last ., or ""
<code>com.pilotfish.FileSize</code>	Long/Integer-ish (<code>file.length()</code>)	Size in bytes
<code>com.pilotfish.SourceFile</code>	<code>java.io.File</code>	Source file object
<code>com.pilotfish.PollingCycleID</code>	Integer	Incremented each time a non-empty accepted batch is processed
<code>com.pilotfish.PollingCycleSize</code>	Integer	Number of files in this cycle batch (cycleSize)
<code>com.pilotfish.RelativeSourceFolderPath</code>	String	Parent folder path relative to polling directory (normalized; polling prefix stripped)

Combine mode: Attributes are taken from the **first** file only.

Tokenizer extras: `com.pilotfish.BatchId`, `com.pilotfish.BatchSize`.

File name parsing nuance: Extension split uses `lastIndexOf('.')`. Names like `archive.tar.gz` -> name `archive.tar`, extension `gz`.

7. Dispatch paths

```
processFile
    createTransactionAttributes
    AutoDeleteFileInputStream (unless post-process skipped)
    copy to EIPCacheManager default cache (sized to file length, capped near Integer.MAX_VALUE)
    sendTransactions(cache, attrs)

sendTransactions
    if Tokenizers set (and not sync):
        split -> notifyListeners per token
    else if Synchronous:
        dispatchForSynchronousResponse(stream, attrs.cloneAllForResponse(), timeout)
    else:
        dispatchListeners(stream, attrs.cloneAllForResponse())

cloneAllForResponse() ensures each dispatch gets a full attribute copy suitable for independent
transaction start.
```

8. Polling cycle and quota (worked example)

Configuration: Max Transactions Per Run = 3, Include sub-folders = true.

```
/poll/
a.txt
b.txt
c.txt
d.txt
sub/
  e.txt
```

1. Reset `serializedTxLeft` = 3
2. List `/poll`, accept files until 3 taken -> e.g. **a,b,c** (order depends on listing/sort/reverse iteration)
3. `serializedTxLeft` = 0
4. Recurse into `sub/` -> immediately return because quota is 0 -> **e.txt waits for a later poll**
5. Next poll picks remaining files (if still present and still matching)

With `Combine files = true` and three accepted files, **one** transaction is emitted for that directory batch (quota still consumed per accepted file before combine).

9. Known quirks and caveats (from source)

1. **Post-process timing:** Delete/Move happens when the listener finishes reading into cache, not when the route succeeds. Failed routes do not restore deleted files.

2. **Keep vs Delete vs unconfigured:** Prefer explicitly setting Keep/Delete/Move; unconfigured empty behaves like delete-oriented stream setup (`keep=false`, no move dir).
3. **Read-only files are ignored** even if you only need to read them – `gpmAccept` requires write permission.
4. **Unreadable file/dir aborts the directory pass** (`return`), not merely skips that entry.
5. **Remote filesystems:** Min-seconds-since-modified is unreliable across clock skew / delayed mtime updates.
6. **CompatModeMoved migration:** Missing XML element => forced true on load.
7. **failedDeleteFiles:** Intended to skip files that failed prior delete/move; comments note related move helper paths as FIXME / unused in places – do not rely on this as durable “already processed” tracking. Prefer Move/Delete post-process or external archival.
8. **moveFile on DirectoryListener itself** is largely superseded by `AutoDeleteFileInputStream`; leftover code/comments acknowledge inconsistencies.
9. **Synchronous + Max Tx != 1** fails hard each poll until fixed.
10. **Tokenizers + large files** load entire content into memory.
11. **PollingCycleID** increments once per batch of accepted files in a directory pass when `acceptedFiles.size() > 0`, not once per file.
12. **Full path mode** error message may say “polling directory” even when referring to a missing file path.
13. **Extension list** is case-normalized; ensure `Util.getExtension` behavior matches your platform naming.
14. **Scheduler + polling:** Scheduling windows and polling interval interact – the poller may still be scheduled but effective listening depends on scheduler enablement in the broader listener framework.

10. Operational recommendations

Goal	Suggested settings
Stable pickup of growing files	Keep Min seconds since file modified $\geq$ write duration; default 10 is a starting point
Exactly-once style handoff	Prefer Move to an archive/error tree rather than Keep
Predictable order	Sorting Method = Modified Time; validate reverse-iteration effect in a test folder
Back-pressure into filesystem	Lower Max Transactions Per Run; enable listener throttling if stage pools saturate
Request/response over files	Synchronous = true, Max Tx = 1, no tokenizers
CSV family with headers	Combine files + Number of header lines
Deep trees	Include sub-folders + Full folder path restriction regex
Legacy routes after upgrade	Inspect CompatModeMoved explicitly after first 19R1+ save

11. Source index (this WAR)

Topic	Path under war-pipeline/decompiled/eip-26R1.11/
DirectoryListener	com/pilotfish/eip/modules/file/DirectoryListener.
AutoDeleteFileInputStream	com/pilotfish/eip/modules/file/AutoDeleteFileInput
AbstractListener	com/pilotfish/eip/extend/AbstractListener.java
AbstractConfiguration tags	com/pilotfish/eip/extend/AbstractConfiguration.ja
Catalog JSON	war-pipeline/index/eip-26R1.11/directory-listener-

Comparison doc (Git-sourced, unchanged): PilotFish-Directory-Listener-Reference.pdf

12. Pipeline provenance

PilotFish WARs/eip.war.hs.26R1.11
-> extract/eip-26R1.11/
-> jars: xcs-modules-file-1.0.1.jar + xcs-core-1.2.6.jar
-> decompiled/eip-26R1.11/ (CFR 0.152)
-> index/eip-26R1.11/directory-listener-catalog.json
-> this deep-dive Markdown/PDF

Deep-dive documentation from WAR decompile – 26R1.11 – July 2026.